AIC_G5T1

Konstantin Unterweger, Jaime Gallego, Benedikt Eckerstorfer, and Marta Volpini

Technische Universität Wien, Vienna, Austria
e12222196@student.tuwien.ac.at, e12505274@student.tuwien.ac.at, e12217063@student.tuwien.ac.at, e12502974@student.tuwien.ac.at

Abstract. This paper presents the design and implementation of a real-time Taxi Fleet Monitoring System, developed to address the challenges of processing high-velocity geospatial telemetry data. Leveraging the T-Drive dataset, the system implements a distributed stream-processing pipeline comprising Apache Kafka for reliable data ingestion, Apache Storm for low-latency event processing, and Redis for state management. Key contributions include a hybrid processing topology capable of stateless velocity calculation and stateful windowed aggregation, as well as a reactive dashboard that visualizes fleet movements and alerts via WebSockets. The evaluation validates the system's scalability and responsiveness, demonstrating the effectiveness of the decoupled producer-broker-processor pattern for spatially distributed IoT applications.

Keywords: Stream processing · Apache Storm · Distributed systems · Real-time monitoring · Trident · Geospatial data

1 Responsibilities and Workload Distribution

This project, the *Taxi Fleet Monitoring System*, required a multidisciplinary approach combining data engineering, distributed systems, and full-stack development. The workload was distributed not only to ensure a separation of concerns but also to allow each member to tackle specific technical challenges relevant to their academic interests.

1.1 Project Scope and Team Organization

The objective was to ingest real-time GPS telemetry from the T-Drive dataset [14, 15], process it via an Apache Storm topology to detect speed violations, and visualize the fleet status on a dynamic dashboard. To manage this complexity, the workload was distributed to ensure a clear separation of concerns, allowing parallel development of the ingestion pipeline (Producer), the processing core (Storm), and the presentation layer (Web/API).

1.2 Team Profiles and Contributions

Jaime Gallego (Backend & System Integration)

- **Background:** Coming from a Telecommunications Engineering background with a specialization in Computer Science, Jaime has strong fundamentals in network protocols and Python development.
- **Focus & Contribution:** He took ownership of the core processing logic and system integration. His primary contribution was the implementation of the Apache Storm topology in Java, translating the theoretical concepts of stateful stream processing (e.g., tumbling windows) into working code. Additionally, he developed the FastAPI backend to bridge the gap between the high-speed processing layer and the user interface.
- **Learning Objective:** To transition from Python-based scripting to building robust, typed distributed applications in Java, specifically managing state and synchronization in an asynchronous environment.

Konstantin Unterweger (Data Ingestion)

- **Background:** With a focus on Data Science, Konstantin is experienced in handling large datasets and analytical pipelines, primarily using Python.
- **Focus & Contribution:** He was responsible for the data supply chain. He implemented the `bulk_producer` to accurately simulate real-time streams from the static T-Drive dataset and configured the Apache Kafka cluster (KRaft mode). His work ensured that the topology received a reliable, time-ordered stream of events.
- **Learning Objective:** To understand the operational intricacies of message brokers and how to implement backpressure mechanisms in ingestion pipelines effectively.

Marta Volpini (Frontend & Visualization)

- **Background:** Sharing a similar academic background in Telecommunications Engineering with Jaime, Marta possesses strong knowledge of communication protocols and signal processing.
- **Focus & Contribution:** She chose to focus on the presentation layer to complement her engineering skills. She led the visualization efforts, engineering the WebSocket client to consume real-time alerts efficiently and designing the dynamic dashboard with Leaflet.js. Her role ensured that the complex backend data was rendered intelligibly for the end-user.
- **Learning Objective:** To broaden her engineering profile towards a "Full Stack" capability by mastering reactive frontend frameworks and real-time data visualization techniques.

Benedikt Eckerstorfer (Infrastructure & Orchestration)

- **Background:** A Computer Science student with solid knowledge of Linux systems and standard containerization practices.
- **Focus & Contribution:** Benedikt managed the deployment and stability of the system. While familiar with Docker, the challenge here was orchestrating a multi-component stack (Zookeeper, Kafka, Storm) with strict networking requirements. He ensured reproducible deployments and configured the environment for the benchmarking phase.
- **Learning Objective:** To deepen his understanding of "Infrastructure as Code" by managing the networking and resource constraints of a distributed cluster, ensuring consistent performance for throughput testing.

1.3 Collaboration Methodology

The team adopted an agile workflow using Git feature branches. To facilitate parallel development, we established a "Contract-First" approach, defining JSON data schemas for Kafka and Redis early in the process. This allowed the Backend, Ingestion, and Frontend components to be developed independently and integrated smoothly for the Release Candidate.

2 Design Decisions and Technology Choice

This project implements a real-time stream-processing system for monitoring a spatially distributed taxi fleet. The overall architecture follows a producer–broker–processor–storage pattern that is commonly used in industrial stream-processing systems [5].

2.1 Technology Stack and Rationale

The data producer was implemented in Python and is responsible for replaying taxi trajectory data into Apache Kafka according to their original timestamps. Although Python is generally slower than compiled languages, this limitation is not critical in this context, as the producer is not part of the latency-sensitive processing path. Its primary responsibilities are correct temporal replay, configurable submission speed, and reliable event emission. Python's strong ecosystem for data processing and file I/O makes it particularly suitable for parsing and preprocessing large trajectory datasets, which is reflected in its widespread use in data engineering, machine learning, and artificial intelligence workflows [11].

Apache Kafka serves as the messaging backbone between the data producer and the stream-processing topology. Kafka was chosen due to its high throughput, durability, and fault tolerance, as well as its ability to decouple data ingestion from downstream processing. Kafka's log-based architecture allows producers and consumers to scale independently while ensuring reliable message delivery [5].

The stream-processing topology was implemented using Apache Storm, with all operators written in Java. Java was selected because it is Apache Storm's primary and most mature programming language, and Storm's core APIs and execution model are designed around it [3]. Using Java avoids the limitations and overhead associated with alternative language bindings and ensures predictable runtime behavior for long-running, stateful operators.

Redis is used as an in-memory data store to maintain state and store aggregated results such as per-taxi distances, average speeds, and current locations. Redis was selected due to its low-latency data access and simple key–value data model, which is well suited for frequently updated dashboard metrics [13].

Dashboard Backend/Frontend and WebSocket Notifications In addition to the stream-processing pipeline, the system includes a web-based dashboard consisting of a frontend and a backend service. The frontend is implemented using standard web technologies (HTML and client-side JavaScript) and focuses exclusively on visualization and user interaction. The backend is implemented in Python and acts as an API layer between Redis and the dashboard.

This separation improves scalability and maintainability: the backend encapsulates data access and aggregation logic, while the frontend remains lightweight and can be scaled independently (e.g., by serving static assets via a web server or CDN). The backend can also be replicated if the number of dashboard clients increases, withou modifying the stream-processing topology.

To support real-time updates and incidents (e.g., speeding or area violations), the backend pushes notifications to the frontend via a WebSocket connection. This avoids inefficient client-side polling and reduces end-to-end latency for alerts. The backend consumes the relevant updates from Redis and forwards them immediately to connected clients, ensuring that the dashboard reflects the latest system state with minimal delay.

2.2 Stability and Maturity

The selected technology stack can be considered mature and battle-tested. Apache Kafka and Apache Storm have been deployed in production stream-processing systems for many years and are supported by extensive documentation and large user communities [5, 10]. Their long-term industrial usage indicates stable APIs and predictable behavior under load.

Python is a well-established programming language with a strong focus on data processing and analytics. Its dominance in data-centric domains such as machine learning and artificial intelligence demonstrates its maturity and suitability for preprocessing and ingestion tasks, even when raw execution speed is not the primary concern [11]. Java remains a standard choice for long-running distributed systems and further contributes to the robustness of the processing layer.

2.3 Risks and Weaknesses

Despite their maturity, the chosen technologies introduce several risks. Operating Apache Kafka, Apache Storm, and Redis as separate distributed systems increases deployment and operational complexity. Configuration, monitoring, and debugging require careful coordination across components.

Apache Storm exposes a relatively low-level programming model and does not natively support exactly-once processing semantics, which may lead to duplicate processing under failure conditions [3]. While acceptable for dashboard-oriented metrics, this limits the applicability of the architecture for use cases requiring strict consistency guarantees.

Redis, as an in-memory data store, is constrained by available memory and prioritizes performance over durability. Without additional persistence mechanisms, stored state may be lost in the event of failures [13]. Finally, Python limits the maximum achievable ingestion throughput of the producer. While sufficient for controlled dataset replay, higher ingestion rates would require horizontal scaling or a reimplementation in a compiled language.

The additional backend–frontend split and WebSocket channel also introduce complexity. Stateful WebSocket connections require connection management and error handling (reconnects, timeouts), and the backend becomes an additional component that must be monitored and scaled. If Redis is used as the sole source of truth for live updates, careful key design and update strategies are required to prevent stale dashboard state.

2.4 Alternatives

More integrated stream-processing frameworks such as Apache Flink were considered as alternatives to Apache Storm. These frameworks provide higher-level abstractions and stronger consistency guarantees but were not chosen in order to closely follow the assignment architecture and to gain experience with low-level stream-processing concepts [2] [9].

3 Evaluation and Analysis

3.1 Experimental Setup

Experiments were conducted on a Windows host with AMD Ryzen 9 5900X (12 cores, 24 threads) and 32 GB RAM. Apache Kafka was configured with 12 topic partitions to enable parallel consumption. Topic partitions are the fundamental unit of parallelism in Kafka, and increasing the number of partitions generally improves achievable throughput by allowing more consumers to process records concurrently [6]. Apache Storm (configurable executors per bolt), Redis 7-alpine and a custom producer feeding Beijing taxi GPS data from historical trajectory files (comprised of up to 10,000 .txt files) completed the architecture. All components ran as Docker containers communicating via Docker's virtual network. The producer allowed configurable pacing, with pace 0 representing maximum throughput and values > 0 simulating realistic arrival patterns.

3.2 Methodology

Two primary test phases were performed:

Phase 1 – Load Progression: Four load levels (500, 2,500, 5,000, 10,000 files) were tested to identify saturation. Metrics for per-operator execute latency and capacity were recorded from Storm UI after a 90-second stabilization period. Capacity values approaching 1.0 indicate saturation [5].

Phase 2 – Parallelism Scaling: Three executor configurations (1, 2, 12 per bolt) were tested under fixed high load (5,000 files, pace 0) to evaluate scaling efficiency. Each configuration involved deleting caches and rebuilding the application.

3.3 Results and Analysis

Load Progression: Table 1 shows per-operator metrics at increasing load levels. The system remains stable up to approximately 400 msgs/sec. RedisBolt capacity reaches 0.975 at 5,000 files and exceeds 1.0 at 10,000 files, confirming it as the primary bottleneck. Other bolts remain well below saturation, indicating severe pipeline imbalance.

Bolt / Metric	**500**	**2,500**	**5,000**	**10,000**
Execute Latency (ms):				
ParseJsonBolt	0.303	0.062	0.040	0.046
SpeedBolt	0.036	0.020	0.026	0.021
AverageSpeedBolt	0.103	0.030	0.019	0.033
RedisBolt	2.019	1.744	2.887	2.887
Capacity:				
ParseJsonBolt	0.006	0.019	0.033	0.047
SpeedBolt	0.001	0.003	0.032	0.021
AverageSpeedBolt	0.001	0.005	0.015	0.084
RedisBolt	0.029	0.141	0.975	**1.025**

Table 1. Per-operator metrics across load progression tests (2 executors). RedisBolt exhibits the highest latency and capacity at all load levels, confirming it as the bottleneck.

Parallelism Scaling: Throughput improves from 149 msgs/sec (1 executor) to 408 msgs/sec (12 executors), but with diminishing returns. RedisBolt remains saturated across all configurations, and its latency increases with parallelism due to contention for Redis resources. This indicates an I/O-bound bottleneck rather than a CPU limitation.

Bolt / Metric	1 Executor	2 Executors	12 Executors
ParseJsonBolt Latency (ms)	0.008	0.012	0.049
SpeedBolt Latency (ms)	0.004	0.009	0.023
AverageSpeedBolt Latency (ms)	0.008	0.010	0.019
RedisBolt Latency (ms)	0.655	0.716	2.816

Table 2. Parallelism scaling: Execute latency per Bolt (ms) for increasing number of executors. RedisBolt latency increases due to contention.

Bolt / Metric	1 Executor	2 Executors	12 Executors
ParseJsonBolt Capacity	0.016	0.034	0.046
SpeedBolt Capacity	0.010	0.025	0.032
AverageSpeedBolt Capacity	0.016	0.022	0.026
RedisBolt Capacity	0.957	1.005	0.966
Throughput (msgs/sec)	149	279	408
Speedup vs. Baseline	1.00x	1.87x	2.74x

Table 3. Parallelism scaling: Capacity and overall throughput for increasing number of executors. RedisBolt remains saturated, limiting speedup.

Discussion and Proposed Solutions:

RedisBolt is the primary bottleneck, consuming >90% of processing time at high load. The saturation and increased latency under parallelism indicate I/O contention rather than CPU limitations. Potential solutions include:

- Redis pipelining or batch writes to reduce per-message overhead [12][7].
- Sharding data across multiple Redis instances to allow parallel writes [7].
- Increasing Redis memory and using optimized data structures to improve throughput.
- Introducing asynchronous writes or caching intermediate results to reduce Redis dependency.

Addressing these I/O limitations is expected to increase throughput beyond the current 400 msgs/sec ceiling and reduce execution latency for RedisBolt. As pointed out by Abhinav Thakur [1], although Redis is highly optimized, under extreme conditions, even it may be insufficient.

Docker limitations: Another potential bottleneck could be communication between Docker containers, as proposed by Anil Kumar Moka [8]. Detailed testing of this is beyond the scope of this course.

4 Conclusion

This project implemented a distributed real-time data processing system for analyzing taxi trajectories using Apache Kafka, Apache Storm, and Redis, orchestrated through Docker Compose. The system ingests historical taxi data,

streams it through Kafka, and processes it in real time using a Storm topology, with results stored in Redis and visualized through a web-based interface. Overall, the system fulfilled the objectives of the assignment and demonstrated how a producer–broker–processor architecture can be applied to real-time geospatial monitoring scenarios.

Throughout the project, we gained valuable insights into the practical challenges of stream processing and distributed system design. Working with several technologies that were new to most team members required a significant learning effort at the beginning, but also became one of the most interesting aspects of the exercise. The project highlighted the importance of understanding how different components interact within a streaming pipeline, rather than focusing on individual technologies in isolation.

Several challenges were encountered during development and deployment. Getting all services to work together reliably at the beginning required careful configuration of Docker Compose, especially with respect to service dependencies and startup order. Ensuring consistent behavior across different operating systems also proved challenging, as differences between Windows and Linux environments caused issues related to Docker networking, file system paths, and port availability. In particular, some team members experienced port conflicts on Windows, where required ports were already in use, preventing containers from starting correctly. Handling data-specific edge cases, such as missing initial state for individual taxis, also required explicit logic to avoid incorrect computations.

Beyond technical challenges, coordination within a team of four members played an important role in the project. As the team members did not previously know each other, both online and in-person meetings were necessary to establish effective communication and collaboration. Regular updates and discussions helped divide the workload, align design decisions, and integrate independently developed components more smoothly, ultimately improving the overall development process.

In summary, the exercise provided a realistic and hands-on introduction to modern stream-processing systems. As constructive feedback, the assignment required working with technologies that had not been previously covered in lectures, making the learning process largely self-directed. While this initially increased the difficulty of the task, it ultimately encouraged deeper exploration of documentation, experimentation, and independent problem-solving. In this sense, the exercise was demanding but rewarding, as it closely resembled real-world engineering scenarios where learning new technologies autonomously is often necessary.

5 Theory Questions

5.1 Architectural Problems and Redundancies

The current architecture appears over-engineered relative to the simplicity of the implemented workload. Kafka is configured in KRaft mode and already pro-

vides buffering, durability, and replay capabilities, making Zookeeper deployment unnecessary, as it is included only to satisfy Apache Storm's coordination requirements [4].

Additionally, a full Apache Storm cluster (Nimbus, Supervisor, and UI) is deployed to execute a single topology whose logic is limited to parsing messages, applying simple processing algorithms, and writing the results to Redis. This logic does not benefit from Storm's distributed scheduling or fault-tolerance mechanisms and introduces unnecessary complexity.

In the alternative architecture, the ingestion and broker layers remain unchanged, with the feeder publishing data to Kafka, which continues to buffer messages. The processing layer replaces the entire Storm and Zookeeper stack with a single Kafka consumer service running in one container. This consumer joins a consumer group, reads from the taxi-locations topic, applies the required processing algorithms locally, and writes the results directly to Redis. The storage and presentation layers remain unchanged, preserving the original data flow while significantly reducing architectural complexity.

5.2 Why Storm Cannot Achieve Exactly-Once Semantics

Standard Apache Storm cannot provide true exactly-once processing semantics because it is fundamentally based on an at-least-once acknowledgment model. In this model, message delivery is guaranteed, but duplicate processing is possible in the presence of failures [3].

When a failure occurs, the tuples may be replayed even if their processing side effects have already been applied. For example, a Bolt may successfully perform an external action, such as writing to a database, but crash before sending the acknowledgment back to the Spout.

As a result, the Spout will replay the tuple, causing the same operation to be executed again. Since Core Storm does not track transaction identifiers or state versions across replays, it cannot detect or prevent duplicate side effects, making exactly-once guarantees impossible without additional coordination mechanisms.

5.3 How Trident Supports Exactly-Once Semantics

Trident addresses Storm's lack of exactly-once semantics by introducing micro-batch processing combined with transactional state management. Instead of processing individual tuples, Trident groups tuples into small batches and assigns each batch a unique transaction identifier. When a batch is replayed after a failure, it is sent with the same identifier, allowing the system to recognize duplicates. State updates in Trident are performed atomically with the corresponding transaction identifier [3].

Before applying an update, Trident compares the incoming transaction ID with the one stored in the state. If the batch has already been processed, the update is skipped; otherwise, it is applied and the new transaction ID is stored. This mechanism allows Trident to detect and ignore replays, thereby providing exactly-once semantics at the state level.

References

1. Abhi Strike: How redis stays so fast, even with just one thread. https://medium.com/@abhi.strike/how-redis-stays-so-fast-even-with-just-one-thread-6aa79235ea8f, accessed: 2026-01-11
2. Apache Software Foundation: Apache flink: Stateful computations over data streams. https://flink.apache.org/, accessed: 2026-01-10
3. Apache Software Foundation: Apache storm documentation. https://storm.apache.org/, accessed: 2026-01-10
4. Apache Software Foundation: Kafka raft (kraft) mode. https://kafka.apache.org/documentation/kraft, accessed: 2026-01-11
5. Apache Software Foundation: Kafka streams architecture. https://kafka.apache.org/10/streams/architecture/, accessed: 2026-01-10
6. Bejeck, B.: Kafka Streams in Action: Real-time apps and microservices with the Kafka Streams API. Manning (2018)
7. Carlson, J.: Redis in Action. Manning (2013)
8. DZone: Docker performance optimization: Real-world strategies. https://dzone.com/articles/docker-performance-optimization-strategies, accessed: 2026-01-11
9. Hirzel, M., Soulé, R., Gedik, B., Grimm, R.: A catalog of stream processing optimizations. ACM Computing Surveys **46**(4) (2014)
10. Parse.ly: Storm at parse.ly. https://www.parse.ly/storm/ (2016), accessed: 2026-01-10
11. Python Software Foundation: Python for data science. https://www.python.org/about/success/, accessed: 2026-01-10
12. Redis Labs: Redis pipelining. https://redis.io/docs/manual/pipelining/, accessed: 2026-01-11
13. Redis Ltd.: Redis documentation. https://redis.io/docs/, accessed: 2026-01-10
14. Yuan, J., Zheng, Y., Xie, X., Sun, G.: Driving with knowledge from the physical world. In: The 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'11). ACM, New York, NY, USA (2011)
15. Yuan, J., Zheng, Y., Zhang, C., Xie, W., Xie, X., Sun, G., Huang, Y.: T-drive: driving directions based on taxi trajectories. In: Proceedings of the 18th SIGSPATIAL International Conference on Advances in Geographic Information Systems (GIS '10). pp. 99–108. ACM, New York, NY, USA (2010)